

Ruby

We will use a Ruby client example to demonstrate that our installation of RabbitMQ is working fine. The Ruby Version Manager (RVM) will be used to install Ruby as shown below:

```
- name: Ruby client
  hosts: rabbitmq
  gather_facts: true
  tags: [ruby]

tasks:
  - name: Import key
    command: gpg2 --keyserver hkp://keys.gnupg.net --recv-
keys 409B6B1796C275462A1703113804BB82D39DC0E3

  - name: Install RVM
    shell: curl -sSL https://get.rvm.io | bash -s stable

  - name: Install Ruby
    shell: source /etc/profile.d/rvm.sh && rvm install ruby-
2.2.6

  - name: Set default Ruby
    command: rvm alias create default ruby-2.2.6

  - name: Install bunny client
    shell: gem install bunny --version ">= 2.6.4"
```

After importing the required GPG keys, RVM and Ruby 2.2.6 are installed on the CentOS 6.8 VM. The *bunny* Ruby client for RabbitMQ is then installed. The Ansible playbook to set up Ruby is given below:

```
$ ansible-playbook -i inventory/kvm/inventory playbooks/
configuration/rabbitmq.yml --tags "ruby"
```

We shall create a 'temperature' queue to send the values in Celsius. The *consumer.rb* code to receive the values from the queue is given below:

```
#!/usr/bin/env ruby

require "bunny"

conn = Bunny.new(:automatically_recover => false)
conn.start

chan = conn.create_channel
queue = chan.queue("temperature")

begin
  puts "... waiting. CTRL+C to exit"
```

```
queue.subscribe(:block => true) do |info, properties, body|
  puts "Received #{body}"
end
rescue Interrupt => _
  conn.close

exit(0)
end
```

The *producer.rb* code to send a sample of five values in degree Celsius is as follows:

```
#!/usr/bin/env ruby

require "bunny"

conn = Bunny.new(:automatically_recover => false)
conn.start

chan = conn.create_channel
queue = chan.queue("temperature")

values = ["33.5", "35.2", "36.7", "37.0", "36.4"]

values.each do |v|
  chan.default_exchange.publish(v, :routing_key => queue.
name)
end
puts "Sent five temperature values."

conn.close
```

As soon as you start the consumer, you will get the following output:

```
$ ruby consumer.rb
... waiting. CTRL+C to exit
```

You can then run the *producer.rb* script that writes the values to the queue:

```
$ ruby producer.rb

Sent five temperature values.
```

The received values at the consumer side are printed out as shown below:

```
$ ruby consumer.rb
... waiting. CTRL+C to exit
Received 33.5
Received 35.2
Received 36.7
```